

# Project 3: Building Arithmetic Routines on a Simulated Computer

Andre Llaneta, Damiane Kapanadze, Hugh Richards

Spring 2026

## Introduction

This project studies how higher-level arithmetic can be built from a very small instruction set on a simulated computer. Instead of using Python's built-in arithmetic directly, we constructed instruction-level routines that operate on a memory array and a single working register, following the architectural model introduced in class. The main objective was to show that even on a constrained machine, arithmetic tasks can be implemented modularly and then composed into a larger program.

The project is organized into five Python files with distinct roles. The file `cpu_core.py` defines the machine model, bit-conversion utilities, instruction encoding, and the CPU execution loop. The files `multiply.py`, `gcd.py`, and `divide.py` each emit a block of machine instructions into memory corresponding to each arithmetic routine. Finally, `main.py` initializes the shared data, writes the top-level control program, places the three arithmetic routines at fixed memory addresses, and then runs the simulated CPU on the fully composed memory routine.

The main demonstration computes

$$\frac{30 \times 18}{\gcd(630, 90)}.$$

This expression was selected because it requires all three routines. The multiplication routine computes the numerator, the GCD routine computes the denominator, and the division routine combines the two intermediate values into the final result. The arithmetic is easy to verify by hand:

$$30 \times 18 = 540, \quad \gcd(630, 90) = 90, \quad \frac{540}{90} = 6.$$

A major part of the project was making the arithmetic work under the same restrictions as the simulated computer. The routines could not rely on built-in multiplication, division, function calls, a stack, or multiple registers. Instead, each routine had to be written in terms of the available instructions, with inputs and outputs passed through agreed-upon memory locations and control passed explicitly using branch instructions.

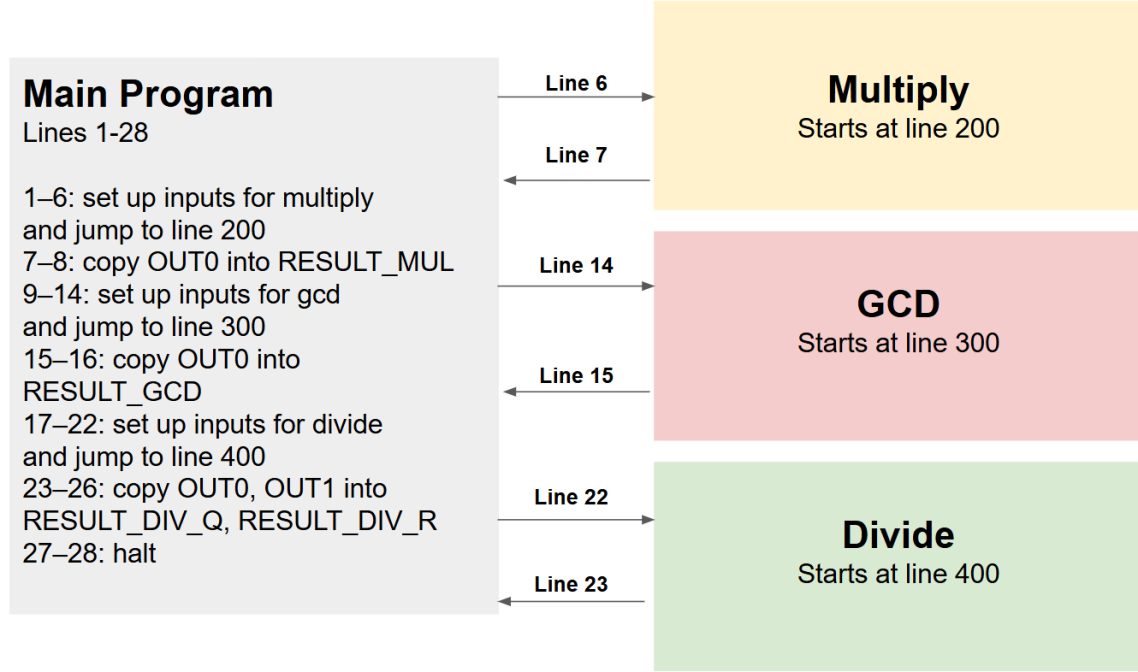


Figure 1: Memory layout of the composed program. The main program is allocated at addresses 1-28, while the multiply, GCD, and divide routines are emitted at fixed addresses. Control flow moves between these blocks using BNZ (branch not zero) instructions and fixed return lines.

## Governing Algorithm and Machine Model

### Machine state

The simulated computer has three main pieces of state:

$$\text{mem} \in \{0, 1\}^{2^{13} \times 16}, \quad \text{reg} \in \{0, 1\}^{16}, \quad pc \in \{0, 1, \dots, 2^{13} - 1\}.$$

Here  $\text{mem}$  is the central memory,  $\text{reg}$  is the single working register, and  $pc$  is the program counter. Each memory word contains 16 bits. When a memory word is interpreted as an instruction, the first 3 bits represent the opcode and the remaining 13 bits represent the associated memory address.

The instruction set used in this project is

LOAD, STORE, ADD, BNZ, AND, SHL, SHR, SUB.

### Instruction procedure

If the instruction at line  $pc^n$  has opcode  $op$  and address  $\text{addr}$ , then one CPU step updates the machine state according to the corresponding instruction rule.

For LOAD,

$$\text{reg}^{n+1} = \text{mem}^n(\text{addr}), \quad \text{mem}^{n+1} = \text{mem}^n.$$

For STORE,

$$\text{mem}^{n+1}(\text{addr}) = \text{reg}^n, \quad \text{reg}^{n+1} = \text{reg}^n.$$

For ADD, interpreting both operands as signed 16-bit 2's complement integers,

$$\text{reg}^{n+1} = \text{reg}^n + \text{mem}^n(\text{addr}) \pmod{2^{16}}.$$

For SUB,

$$\text{reg}^{n+1} = \text{reg}^n - \text{mem}^n(\text{addr}) \pmod{2^{16}}.$$

For AND,

$$\text{reg}^{n+1} = \text{reg}^n \wedge \text{mem}^n(\text{addr}).$$

For SHL,

$$\text{reg}^{n+1} = \text{reg}^n \ll \text{mem}^n(\text{addr}),$$

with the result truncated back to 16 bits.

For SHR,

$$\text{reg}^{n+1} = \text{reg}^n \gg \text{mem}^n(\text{addr}),$$

where the right shift is arithmetic, so the sign bit is preserved for negative numbers in 2's complement.

For BNZ,

$$pc^{n+1} = \begin{cases} \text{addr}, & \text{if } \text{reg}^n \neq 0, \\ pc^n + 1 \pmod{2^{13}}, & \text{otherwise.} \end{cases}$$

For all non-branch instructions, the program counter advances by one after the instruction is executed.

## Shared memory convention

Since the machine has no stack and no true subroutine mechanism, all routines must agree on where inputs, outputs, constants, and temporary values live. The main shared locations are

ARG0, ARG1 for routine inputs,

OUT0, OUT1 for routine outputs.

There are also fixed constants such as ZERO, ONE, SIGN\_MASK, and CONST\_16, along with routine-specific working locations such as MUL\_A, MUL\_B, GCD\_WORK, and DIV\_R.

The top-level program in `main.py` computes the final expression in three stages:

1. place  $a$  and  $b$  into ARG0, ARG1 and jump to the multiplication routine at line 200;
2. place  $c$  and  $d$  into ARG0, ARG1 and jump to the GCD routine at line 300;
3. place the multiplication result and GCD result into ARG0, ARG1 and jump to the division routine at line 400.

Each routine returns by branching to a fixed continuation line in the main program.

## Two's Complement Representation

Because the simulated machine lacks separate instructions for signed and unsigned operations, all negative numbers are natively handled using 2's complement representation. In a 16-bit architecture, the most significant bit (MSB) serves as the sign bit. A 0 indicates a positive number, while a 1 indicates a negative number, yielding a representable integer range from  $-2^{15}$  to  $2^{15} - 1$ .

To form the negative of a binary number in 2's complement, all bits are inverted and 1 is added to the result. This representation is vital for the multiplication routine. Booth's algorithm inherently supports negative multipliers and multiplicands precisely because 2's complement allows subtraction to be executed using the same binary logic as addition. Additionally, because the `SHR` instruction performs an arithmetic right shift, it automatically duplicates the sign bit. This preserves the correct 2's complement value of negative numbers as the multiplier is shifted down during the execution loop, avoiding the need for dedicated sign-checking logic.

## Multiplication routine

The multiplication routine computes

$$\text{OUT0} \leftarrow \text{ARG0} \times \text{ARG1}$$

using a Booth-style multiplication algorithm. Instead of adding the multiplicand once for every 1-bit in the multiplier, Booth multiplication looks for transitions in the multiplier bit pattern. This is useful because a long run of 1s can be represented more efficiently as the difference between two powers of two.

The routine begins by copying `ARG0` and `ARG1` into local working storage `MUL_A` and `MUL_B`. At each loop step, the routine extracts the current least significant bit of the multiplier and compares it with the previous bit. The pair is interpreted as

(current bit, previous bit).

The update rule is shown in Table 1.

| Current bit | Previous bit | Pattern | Action                |
|-------------|--------------|---------|-----------------------|
| 0           | 1            | 01      | Add $A$ to $P$        |
| 1           | 0            | 10      | Subtract $A$ from $P$ |
| 0           | 0            | 00      | Do nothing            |
| 1           | 1            | 11      | Do nothing            |

Table 1: Booth multiplication update rule based on the current and previous multiplier bits.

The routine only changes the product when the multiplier enters or leaves a run of 1s. A transition from 0 to 1 begins a run and causes a subtraction, while a transition from 1 to 0 ends a run and causes an addition of the appropriately shifted multiplicand. After this decision, the routine updates the previous bit, left-shifts the multiplicand, arithmetic-right-shifts the multiplier, decrements the loop counter, and repeats.

For example, consider

$$37 \times 15.$$

The multiplier 15 is

$$15 = (0000\ 0000\ 0000\ 1111)_2.$$

A normal shift-add multiplier would add 37, 74, 148, and 296, because there are four 1-bits. Booth multiplication treats the run of four 1s more compactly:

$$15 = 16 - 1.$$

So the product can be computed as

$$37 \times 15 = 37(16 - 1) = 592 - 37 = 555.$$

In the routine, this corresponds to subtracting 37 when the run of 1s begins, doing no arithmetic update for the middle of the run, and then adding  $37 \ll 4 = 592$  when the run ends. This is why Booth multiplication is especially efficient for dense multiplier patterns with long runs of 1s, while alternating-bit patterns are less favorable because they create many transitions.

The routine returns the final product in `OUT0`.

## GCD routine

The GCD routine computes

$$\text{OUT0} \leftarrow \text{gcd}(\text{ARG0}, \text{ARG1})$$

using Euclid's algorithm. The inputs are assumed to be nonnegative.

The routine first copies the inputs into local working storage `GCD_A` and `GCD_B`. It then repeatedly applies

$$(A, B) \mapsto (B, A \bmod B)$$

until  $B = 0$ . Since the instruction set has no modulo instruction, the remainder  $A \bmod B$  is computed by repeated subtraction. A working copy `GCD_WORK` is initialized to  $A$ , and  $B$  is repeatedly subtracted until the next subtraction would produce a negative value. Negativity is detected by masking the sign bit using

$$\text{AND SIGN\_MASK}, \quad \text{SIGN\_MASK} = 2^{15} = 0x8000.$$

The value remaining in `GCD_WORK` is then the remainder.

The routine then updates  $A \leftarrow B$ ,  $B \leftarrow A \bmod B$ , and repeats until  $B = 0$ . The final answer is stored in `OUT0`.

## Division routine

The division routine computes integer division for a positive dividend and positive nonzero divisor:

$$\text{ARG0} \div \text{ARG1} = Q \text{ remainder } R,$$

with

$$\text{OUT0} = Q, \quad \text{OUT1} = R.$$

The outputs satisfy

$$A = BQ + R, \quad 0 \leq R < B.$$

The routine initializes

$$Q = 0, \quad R = A,$$

using local storage `DIV_Q` and `DIV_R`. It then repeatedly tests whether  $R - B$  would be negative. If not, the routine replaces  $R \leftarrow R - B$  and increments the quotient  $Q \leftarrow Q + 1$ . As in the GCD routine, the sign bit is isolated using `AND SIGN_MASK`. When the next subtraction would become negative, the current quotient and remainder are stored in `OUT0` and `OUT1`.

## Efficiency of repeated subtraction

The GCD and division routines both use repeated subtraction because the simulated computer has no built-in modulo or division instruction. This makes the routines straightforward to express using the available instruction set: each loop only needs instructions such as `LOAD`, `SUB`, `AND`, `BNZ`, and `STORE`. The downside is efficiency. If the dividend is large and the divisor is small, division may require many loop iterations. For example, computing  $1000 \div 2$  by repeated subtraction requires 500 quotient increments.

A more efficient division routine could use a shift-and-subtract method, similar to long division in binary. Instead of subtracting the divisor one copy at a time, the routine would shift the divisor left to find the largest power-of-two multiple that fits inside the current remainder, subtract that larger amount, and add the corresponding power of two to the quotient. This would reduce the number of subtraction steps from roughly proportional to the quotient to roughly proportional to the number of bits.

A similar improvement could be made for the GCD routine. The current Euclidean algorithm is mathematically correct, but its modulo step is implemented by repeated subtraction. A faster version would compute the remainder using a more efficient division-style routine, or use a binary GCD algorithm that relies more heavily on shifts and subtraction. These alternatives would be more complicated to implement on the restricted machine, but they would scale better for larger inputs.

## Numerical Method

Unlike the earlier physical-simulation projects in the course, this project does not involve a continuous-time differential equation or a timestep approximation such as Euler. The machine model is already discrete. Therefore the numerical method is the exact repeated application of the CPU update rules.

If  $(\text{mem}^n, \text{reg}^n, pc^n)$  denotes the machine state after  $n$  executed instructions, then the simulation consists of iterating

$$(\text{mem}^{n+1}, \text{reg}^{n+1}, pc^{n+1}) = \Phi(\text{mem}^n, \text{reg}^n, pc^n),$$

where  $\Phi$  is determined by the opcode and address of the current instruction.

The program counter is incremented before a branch can overwrite it. This matches the architectural logic from the course notes and ensures that `BNZ` behaves correctly. Since the arithmetic routines are emitted into memory by Python functions, the practical numerical method for each routine is its machine-language control flow:

- copy inputs into local memory,
- execute the instruction loop,
- store outputs into shared locations,
- branch back to a fixed return line.

This project is therefore best understood as a discrete dynamical system on machine state rather than a continuous dynamical system in physical time.

# Validation

## Instruction-level checks

The first level of validation was to confirm that the CPU core reproduces the intended instruction outputs. During development, small one-purpose tests were used to check:

- whether LOAD and STORE move data correctly;
- whether ADD and SUB perform 16-bit signed wraparound correctly;
- whether BNZ branches exactly when the register is nonzero;
- whether SHR is arithmetic and therefore preserves the sign bit on negative numbers.

## Multiplication validation

The multiplication routine was tested on a wide range of cases. The expected products were computed independently by ordinary arithmetic. The full set of representative validation cases is shown in Table 2.

Table 2: Representative multiplication test cases.

| Case | A   | B   | Expected Product | Why this case matters                                             |
|------|-----|-----|------------------|-------------------------------------------------------------------|
| M1   | 31  | 31  | 961              | Long run of 1s in multiplier; classic Booth-favorable case        |
| M2   | 37  | 0   | 0                | Zero multiplier                                                   |
| M3   | 37  | 1   | 37               | Identity case                                                     |
| M4   | 37  | 2   | 74               | Single shifted 1-bit                                              |
| M5   | 37  | 3   | 111              | Two adjacent 1s; short run behavior                               |
| M6   | 37  | 7   | 259              | Dense low run of 1s                                               |
| M7   | 37  | 15  | 555              | Longer run of 1s to test Booth recoding advantage                 |
| M8   | 37  | 85  | 3145             | Alternating-bit multiplier $(01010101)_2$ ; unfavorable for Booth |
| M9   | 37  | 170 | 6290             | Alternating-bit multiplier $(10101010)_2$ ; unfavorable for Booth |
| M10  | 37  | 255 | 9435             | Dense multiplier with many 1s                                     |
| M11  | -37 | 85  | -3145            | Negative multiplicand with patterned multiplier                   |
| M12  | -37 | -85 | 3145             | Both operands negative                                            |

## GCD validation

The GCD routine was tested on a wide range of cases. The expected GCD values were computed independently by ordinary arithmetic. The full set of representative validation cases is shown in Table 3.

Table 3: Representative GCD test cases.

| Case | A   | B   | Expected GCD | Why this case matters                        |
|------|-----|-----|--------------|----------------------------------------------|
| G1   | 48  | 18  | 6            | Standard textbook Euclidean example          |
| G2   | 54  | 24  | 6            | Multiple subtraction / swap cycles           |
| G3   | 84  | 126 | 42           | Larger common factor                         |
| G4   | 100 | 25  | 25           | One number divides the other exactly         |
| G5   | 81  | 27  | 27           | Exact divisibility with larger shared factor |
| G6   | 35  | 64  | 1            | Coprime inputs                               |
| G7   | 119 | 544 | 17           | Nontrivial Euclidean reduction               |
| G8   | 252 | 105 | 21           | Multi-step Euclidean sequence                |
| G9   | 144 | 168 | 24           | Composite numbers with repeated factors      |
| G10  | 420 | 378 | 42           | Larger composite pair                        |
| G11  | 7   | 0   | 7            | Base case $\gcd(A, 0) = A$                   |
| G12  | 0   | 13  | 13           | Base case after swap-style reasoning         |

## Division validation

The division routine was validated by checking that the outputs satisfy

$$A = BQ + R, \quad 0 \leq R < B,$$

and by comparing quotient and remainder against Python’s integer arithmetic on positive inputs. The chosen cases include exact division, nonzero remainders, divisor-larger-than-dividend cases, larger quotients, and near-limit values. The full set of representative validation cases is shown in Table 4.



Table 4: Representative division test cases.

| Case | A     | B   | Expected Q | Expected R | Why this case matters                        |
|------|-------|-----|------------|------------|----------------------------------------------|
| D1   | 100   | 7   | 14         | 2          | Baseline repeated-subtraction case           |
| D2   | 48    | 6   | 8          | 0          | Exact divisibility                           |
| D3   | 37    | 5   | 7          | 2          | Small remainder case                         |
| D4   | 31    | 31  | 1          | 0          | Dividend equals divisor                      |
| D5   | 31    | 40  | 0          | 31         | Divisor greater than dividend                |
| D6   | 84    | 13  | 6          | 6          | Nontrivial quotient and remainder            |
| D7   | 81    | 9   | 9          | 0          | Exact divisibility with repeated subtraction |
| D8   | 127   | 8   | 15         | 7          | Near power-of-two divisor                    |
| D9   | 144   | 12  | 12         | 0          | Larger exact quotient                        |
| D10  | 170   | 16  | 10         | 10         | Larger remainder case                        |
| D11  | 255   | 15  | 17         | 0          | Dense positive value, exact divisibility     |
| D12  | 32385 | 255 | 127        | 0          | Near 16-bit positive limit, exact case       |

## Composed-program validation

The strongest validation is the full composed program itself, because it checks whether the routines work under one shared memory convention. For the main example,

$$\frac{30 \times 18}{\text{gcd}(630, 90)} = \frac{540}{90} = 6.$$

The machine should therefore return quotient 6 and remainder 0.

Additional composed examples were also used:

$$\frac{12 \times 14}{\text{gcd}(420, 84)} = \frac{168}{84} = 2,$$

and

$$\frac{255 \times 127}{\text{gcd}(381, 254)} = \frac{32385}{127} = 255.$$

These cases test the fixed-return control flow through all three routines and verify that intermediate values are passed correctly from one stage to the next.

To test the remainder output in the full composed program, we also used a case where the final division is not exact:

$$\frac{25 \times 14}{\text{gcd}(210, 84)}.$$

The expected intermediate values are

$$25 \times 14 = 350, \quad \text{gcd}(210, 84) = 42.$$

The final division is therefore

$$350 \div 42 = 8 \text{ remainder } 14,$$

because

$$350 = 42(8) + 14, \quad 0 \leq 14 < 42.$$

This case is important because it verifies that the composed program correctly passes both outputs of the division routine: the quotient in OUT0 and the remainder in OUT1.

## Interesting Aspects of the Code

### Emitter-based construction

A distinctive feature of the code is that the arithmetic files do not directly perform multiplication, GCD, or division in Python. Instead, each routine is written as an emitter that places a block of machine instructions into the simulated memory. This makes the project closer to assembly-language programming than to ordinary Python programming.

The main program does not multiply two integers by using the Python operator `*`. Instead, it writes the inputs into shared memory, jumps to the multiply routine, waits for that routine to return a value in `OUT0`, stores the result, and then continues.

Listing 1: Top-level control flow from `main.py`

```
1 set_instr(mem, 1, LOAD, A['INPUT_A'])
2 set_instr(mem, 2, STORE, A['ARG0'])
3 set_instr(mem, 3, LOAD, A['INPUT_B'])
4 set_instr(mem, 4, STORE, A['ARG1'])
5 set_instr(mem, 5, LOAD, A['ONE'])
6 set_instr(mem, 6, BNZ, MUL_START)
7
8 set_instr(mem, 7, LOAD, A['OUT0'])
9 set_instr(mem, 8, STORE, A['RESULT_MUL'])
```

### Fixed-return subroutines

Since the machine has no `CALL` or `RET`, the routines simulate subroutine behavior by branching back to fixed continuation lines. The main program jumps to the routine start, and the routine ends by loading `ONE` and branching back to a predetermined line.

### Sign detection without compare

The machine has no direct compare instruction and no branch-on-negative instruction. To detect whether a subtraction result has become negative, the code isolates the sign bit using `AND SIGN_MASK`, where

$$\text{SIGN\_MASK} = 2^{15} = 0x8000.$$

This is used in both the GCD and division routines to stop repeated subtraction at exactly the right point.

Listing 2: Sign-test pattern

```
1 set_instr(mem, pc, LOAD, a['DIV_R']); pc += 1
2 set_instr(mem, pc, SUB, a['ARG1']); pc += 1
3 set_instr(mem, pc, AND, a['SIGN_MASK']); pc += 1
4 done_branch = pc
5 set_instr(mem, pc, BNZ, 0); pc += 1
```

### Instruction counting

The CPU core counts the number of times each opcode is executed. This gives a simple way to compare the computational cost of different routines and different composed examples. Since the purpose of the project

is to understand instruction-level execution, these opcode counts are more meaningful than ordinary Python runtime.

## Results and Discussion

The results are organized around both routine-level tests and three composed case studies. Since the central idea of the project is modular arithmetic on one memory image, the composed cases are the most important results.

### Full program results

**Case 1:**  $\frac{30 \times 18}{\gcd(630, 90)}$

#### Inputs

- Multiply inputs: 30, 18
- GCD inputs: 630, 90

#### Expected values

$$30 \times 18 = 540, \quad \gcd(630, 90) = 90, \quad 540 \div 90 = 6.$$

#### Discussion

This is the baseline composed example because it uses all three routines and gives an exact final quotient with zero remainder. The multiplication stage produces 540, the GCD stage reduces the denominator to 90, and the division stage returns quotient 6. This case demonstrates that the routines work together under the shared memory convention and fixed-return control flow.

```

Composed arithmetic program on the simulated computer
Expression: (30 * 18) / gcd(630, 90)

Multiply result: 540 (expected 540)
GCD result: 90 (expected 90)
Division result: quotient = 6, remainder = 0
Expected: quotient = 6, remainder = 0

LOAD: 214
STORE: 129
ADD: 12
BNZ: 105
AND: 31
SHL: 16
SHR: 16
SUB: 62
Total Instructions: 585

```

Figure 2: Terminal output for Case 1. The multiplication stage gives 540, the GCD stage gives 90, and the final division returns quotient 6 with remainder 0.

**Case 2:**  $\frac{12 \times 14}{\gcd(420, 84)}$

#### Inputs

- Multiply inputs: 12, 14
- GCD inputs: 420, 84

#### Expected values

$$12 \times 14 = 168, \quad \gcd(420, 84) = 84, \quad 168 \div 84 = 2.$$

#### Discussion

This case is smaller than the baseline and is useful because the GCD is large relative to the numerator, so the final quotient is small. It checks that the routines still work when the last division is short. Since 84 divides 168 exactly, the remainder is again zero.

```

Composed arithmetic program on the simulated computer
Expression: (12 * 14) / gcd(420, 84)

Multiply result: 168 (expected 168)
GCD result: 84 (expected 84)
Division result: quotient = 2, remainder = 0
Expected: quotient = 2, remainder = 0

LOAD: 191
STORE: 117
ADD: 5
BNZ: 92
AND: 25
SHL: 16
SHR: 16
SUB: 49
Total Instructions: 511

```

Figure 3: Terminal output for Case 2. The intermediate values are 168 and 84, and the final quotient is 2 with remainder 0.

**Case 3:**  $\frac{255 \times 127}{\gcd(381, 254)}$

#### Inputs

- Multiply inputs: 255, 127
- GCD inputs: 381, 254

#### Expected values

$$255 \times 127 = 32385, \quad \gcd(381, 254) = 127, \quad 32385 \div 127 = 255.$$

#### Discussion

The product 32385 lies close to the largest positive signed 16-bit value 32767, so it checks that the multiplication routine remains correct near the machine limit without overflowing. The final quotient of 255 is exact, so the case remains easy to verify by hand.

```

Composed arithmetic program on the simulated computer
Expression: (255 * 127) / gcd(381, 254)

Multiply result: 32385 (expected 32385)
GCD result: 127 (expected 127)
Division result: quotient = 255, remainder = 0
Expected: quotient = 255, remainder = 0

LOAD: 1203
STORE: 624
ADD: 258
BNZ: 597
AND: 277
SHL: 16
SHR: 16
SUB: 552
Total Instructions: 3543

```

Figure 4: Terminal output for Case 3. This near-limit stress test produces product 32385, GCD 127, and final quotient 255 with remainder 0.

**Case 4:**  $\frac{25 \times 14}{\gcd(210, 84)}$

#### Inputs

- Multiply inputs: 25, 14
- GCD inputs: 210, 84

#### Expected values

$$25 \times 14 = 350, \quad \gcd(210, 84) = 42.$$

Therefore,

$$350 \div 42 = 8 \text{ remainder } 14,$$

since

$$350 = 42(8) + 14, \quad 0 \leq 14 < 42.$$

#### Discussion

This case was added to test the final remainder output of the full composed program. The earlier composed examples all ended with remainder 0, so they verified the quotient output but did not fully test whether

the composed program preserved the nonzero remainder returned by the division routine. In this case, the multiplication stage produces 350, the GCD stage produces 42, and the division stage should return quotient 8 and remainder 14. This confirms that both outputs of the division routine are passed back correctly through the shared memory convention.

```
Multiply result: 350 (expected 350)
GCD result: 42 (expected 42)
Division result: quotient = 8, remainder = 14
Expected: quotient = 8, remainder = 14

LOAD: 218
STORE: 131
ADD: 11
BNZ: 105
AND: 31
SHL: 16
SHR: 16
SUB: 60
Total Instructions: 588
```

Figure 5: Terminal output for Case 4. This composed nonzero-remainder case produces product 350, GCD 42, and final division result quotient 8 with remainder 14.

## Overall discussion

The results show that the project works at two levels. First, the three routines work in isolation on their own. Second, they also work together when assembled into one memory image and executed as one application, which validates the memory convention, the branch-based calling structure, and the correctness of the fixed return lines.

The nonzero-remainder composed case strengthens the validation because it checks both outputs of the division routine in the full program. The earlier composed examples all had final remainder 0, so they did not fully test whether a nonzero value in `OUT1` was preserved and reported correctly after division. Case 4 confirms that the composed program returns both quotient and remainder correctly.

A clear limitation is that both GCD and division use repeated subtraction. This is simple and appropriate for the restricted instruction set, but it becomes inefficient for larger inputs. More efficient versions could use binary shift-and-subtract division or a faster modulo routine inside Euclid's algorithm. These approaches would require more control logic and memory locations, but they would reduce the number of loop iterations for large dividends or large intermediate values.

## Conclusions

This project built a small arithmetic library on top of a simulated computer with one working register, one program counter, and a shared code/data memory. The three main routines were Booth multiplication, Euclidean GCD using repeated-subtraction modulo, and integer division by repeated subtraction. These routines were then composed into one larger program that evaluated full arithmetic expressions from start to finish on the simulated machine.

The validation shows that the routines work both separately and together. The composed examples verify that multiplication, GCD, and division can share memory locations and return control correctly through fixed branch targets. The added nonzero-remainder case strengthens the validation because it checks that the full program returns both outputs of the division routine: the quotient and the remainder.

The main lesson of the project is that arithmetic is not only a mathematical operation but also a programming structure imposed by the architecture. Without true subroutine support, no compare instruction, and no direct modulo or divide instruction, the routines had to be written in a way that made control flow and memory layout explicit. Repeated subtraction made GCD and division possible on the restricted machine, but it also revealed a performance limitation. More efficient shift-based division or modulo routines would be a natural next improvement.

## References

- [1] Charles S. Peskin, *Computer Simulation of a Computer*, Courant Institute of Mathematical Sciences, New York University, May 2025.
- [2] Python Software Foundation, *Python 3 Documentation*. <https://docs.python.org/3/>
- [3] NumPy Developers, *NumPy Documentation*. <https://numpy.org/doc/>



## Appendix: Code Snippets

Listing 3: CPU execution loop from `cpu_core.py`

```
1 while pc > 0:
2     instr = mem[pc]
3     opcode = bin2dec(instr[0:3], 3)
4     address = bin2dec(instr[3:16], 13)
5
6     pc = (pc + 1) % rows
7     data[9] += 1
8
9     match opcode:
10        case 0: # LOAD
11            reg = mem[address].copy()
12        case 1: # STORE
13            mem[address] = reg.copy()
14        case 2: # ADD
15            ...
16        case 3: # BNZ
17            if np.any(reg):
18                pc = address
19            ...
```

Listing 4: Top-level program assembly from `main.py`

```
1 def build_program(mem):
2     write_main(mem)
3     emit_multiply(mem, MUL_START, A, return_line=7)
4     emit_gcd(mem, GCD_START, A, return_line=15)
5     emit_divide(mem, DIV_START, A, return_line=23)
```

Listing 5: Instruction Set from `multiply.py`

```
1 set_instr(mem, pc, LOAD, a['ARG0']); pc += 1
2 set_instr(mem, pc, STORE, a['MUL_A']); pc += 1
3 set_instr(mem, pc, LOAD, a['ARG1']); pc += 1
4 set_instr(mem, pc, STORE, a['MUL_B']); pc += 1
5
6 set_instr(mem, pc, LOAD, a['ZERO']); pc += 1
7 set_instr(mem, pc, STORE, a['MUL_P']); pc += 1
8 set_instr(mem, pc, LOAD, a['CONST_16']); pc += 1
9 set_instr(mem, pc, STORE, a['MUL_CNT']); pc += 1
```

Listing 6: Subtraction loop Instruction Set from `gcd.py` and `divide.py`

```
1 set_instr(mem, pc, LOAD, a['GCD_WORK']); pc += 1
2 set_instr(mem, pc, SUB, a['GCD_B']); pc += 1
3 set_instr(mem, pc, AND, a['SIGN_MASK']); pc += 1
4 mod_done_branch = pc
5 set_instr(mem, pc, BNZ, 0); pc += 1
6
7 set_instr(mem, pc, LOAD, a['GCD_WORK']); pc += 1
8 set_instr(mem, pc, SUB, a['GCD_B']); pc += 1
9 set_instr(mem, pc, STORE, a['GCD_WORK']); pc += 1
```